About this PDF

Built only from: **Theory of Computation (Automata and Formal Languages)** — https://nptel.ac.in/courses/106104028

Licence: CC BY-SA 4.0. Accessed .

Covers 4 of 4 syllabus sub-topics of this unit.

Regular expression

Definition

A regular expression is an algebraic way of describing a language over an alphabet Sigma. The base regular expressions include the empty language phi, the empty string epsilon, and any symbol a from Sigma. Inductive regular expressions are formed using union (R1+R2 or R1|R2), concatenation (R1R2), and Kleene closure (R*). A language is regular if and only if it can be described by a regular expression, or equivalently, if it is accepted by some deterministic or non-deterministic finite automaton.

Explanation

Regular expressions use the base cases and inductive operations to build complex languages from simple ones. The operations correspond to set operations on languages: union produces the set union, concatenation produces all concatenations of strings from the operand languages, and Kleene star produces all finite concatenations (including the empty string). Because every regular expression can be converted into an equivalent finite automaton (via Thompson’s construction) and every finite automaton can be converted back into an equivalent regular expression (via state elimination or Arden’s theorem), regular expressions and finite automata have equivalent descriptive power.

Example

The regular expression `a` denotes the language containing the single string of length one, `epsilon` denotes the language containing only the empty string, and `phi` denotes the empty language.

Algorithm / Steps

1. **Thompson’s construction - base cases**: For phi, epsilon, and a, build an epsilon-NFA with distinct start and accepting states.
2. **Thompson’s construction - inductive operations**: Combine the epsilon-NFAs for R1 and R2 using epsilon-transitions to realize union, concatenation, or Kleene star without introducing cycles into the outer structure.
3. **State elimination**: Remove intermediate states one by one, updating transitions between the remaining states with regular expressions that represent all possible paths through the eliminated state until only the initial and accepting states remain.

Key points

- Regular expressions algebraically describe languages over Sigma using base cases phi, epsilon, and a.
- Inductive operations are union, concatenation, and Kleene star.
- Regular languages are exactly those describable by regular expressions or accepted by a DFA/NFA.
- Thompson’s construction converts any regular expression into an equivalent epsilon-NFA.
- State elimination (or Arden’s theorem) converts a DFA/NFA into an equivalent regular expression.

Regular Languages

Definition

A regular language is a language that can be described by a regular expression, or equivalently, accepted by some deterministic or non-deterministic finite automaton.

Explanation

Regular expressions are built from base expressions-empty language phi, empty string epsilon, and single symbols ainSigma-using union, concatenation, and Kleene star. These operations correspond to union, concatenation, and repetition of strings in the language. Finite automata and regular expressions have the same expressive power: for every regular expression there is an equivalent finite automaton, and for every finite automaton there is an equivalent regular expression. Regular languages are closed under many operations: union, concatenation, Kleene star, complement, intersection, difference, reversal, homomorphism, and inverse homomorphism. These closures can be proved by constructing appropriate automata or using algebraic identities. The pumping lemma provides a necessary condition for regularity; if a language fails the lemma, it is not regular.

Example

The regular expression a denotes the language containing only the single string “a”.

Algorithm / Steps

1. **Convert Regular Expression to NFA (Thompson’s construction)**
   - For each base case (phi, epsilon, a) build a small epsilon-NFA with distinct start and accepting states.
   - For union, concatenation, and Kleene star, combine the smaller automata using epsilon-transitions without creating cycles in the outer structure.
2. **Convert Finite Automaton to Regular Expression (State Elimination)**
   - Remove intermediate states one by one.
   - Update transitions between remaining states with regular expressions that represent all possible paths through the eliminated state.
   - Continue until only the initial and accepting states remain; the resulting expression describes the language.

AcademicsGen — source: NPTEL — CC BY-SA 4.0

3. **Prove a Language is Not Regular (Pumping Lemma)**
   - Assume the language is regular and let p be its pumping length.
   - Choose a string w in the language with |w|>=p.
   - The contradiction shows the language is not regular.

Key points

- Regular languages can be described by regular expressions or accepted by finite automata.
- Regular expressions use union, concatenation, and Kleene star; finite automata are equivalent in expressive power.
- Regular languages are closed under union, concatenation, Kleene star, complement, intersection, difference, reversal, homomorphism, and inverse homomorphism.
- The pumping lemma is a tool to prove that certain languages are not regular.
- Thompson’s construction and state elimination provide systematic conversions between regular expressions and finite automata.

Equivalence of Finite Automata and regular expressions Proving languages to be not regular (Pumping Lemma)

Definition

Finite automata and regular expressions possess equivalent descriptive power. For every regular expression, there exists an equivalent finite automaton, and for every finite automaton, there exists an equivalent regular expression. The Pumping Lemma is a fundamental property of all regular languages used to prove that a specific language is not regular via contradiction. Let L be a regular language. Then there exists a constant integer p (the pumping length) such that any string w in L with length at least p can be divided into three substrings, w = x y z, satisfying |y| > 0, |x y| <= p, and for all i >= 0, x y^i z in L.

Explanation

Converting a regular expression to a finite automaton can be done by Thompson's construction, which builds an equivalent non-deterministic finite automaton with epsilon-transitions. Converting a finite automaton to a regular expression can be achieved using the State Elimination technique or Arden's Theorem, removing intermediate states and updating transitions with regular expressions until only the initial and accepting states remain. The Pumping Lemma provides a necessary condition for regularity: any sufficiently long string in a regular language can be “pumped” by repeating a middle segment without leaving the language. To prove a language is not regular, one assumes it is regular, selects a string of length at least the pumping length, and shows

Closure properties of regular languages.

Definition

Regular languages are closed under a comprehensive collection of fundamental set and language operations. If L1 and L2 are regular languages, then the following operations produce regular languages: union, concatenation, Kleene star, complement, intersection, difference, reversal, homomorphism, and inverse homomorphism.

Explanation

Union: The union of two regular languages L1 union L2 is regular, represented by R1 + R2. Concatenation: The concatenation L1L2 is regular, represented by R1R2. Kleene Star: The Kleene star L1* is regular, represented by R1*. Complement: The complement Sigma* − L1 is regular, proved by interchanging the accepting and non-accepting states of a complete DFA for L1. Intersection: The intersection L1 intersection L2 is regular, proved using De Morgan's Law or by direct Cartesian product DFA construction. Difference: The difference L1 − L2 is regular, since L1 − L2 = L1 intersection (complement of L2). Reversal: The language of reversed strings L1^R is regular, constructed by reversing all transitions of an NFA and swapping start and final states. Homomorphism and Inverse Homomorphism: Substitution of alphabet symbols preserves regularity. Decision properties: Regular languages have effective algorithms to determine emptiness, finiteness, infiniteness, and equivalence of finite automata.

Algorithm / Steps

1. Complement: Interchange accepting and non-accepting states of a complete DFA for L1.
2. Intersection: Use De Morgan's Law L1 intersection L2 = complement of (complement of L1 union complement of L2) or construct a Cartesian product DFA.
3. Difference: Compute L1 intersection (complement of L2).
4. Reversal: Reverse all transitions of an NFA and swap start and final states.
5. Homomorphism: Substitute alphabet symbols; the resulting language remains regular.
6. Inverse Homomorphism: Apply inverse substitution; the resulting language remains regular.

Key points

- Regular languages are closed under union, concatenation, and Kleene star.
- They are also closed under complement, intersection, and difference.
- Reversal, homomorphism, and inverse homomorphism preserve regularity.
- Decision problems such as emptiness, finiteness, infiniteness, and equivalence are effectively solvable for regular languages.

Source and licence

This PDF is adapted from **Theory of Computation (Automata and Formal Languages)** (https://nptel.ac.in/courses/106104028), accessed .

Source licence: **CC BY-SA 4.0** — https://creativecommons.org/licenses/by-sa/4.0/

Changes made by AcademicsGen: the source text was rephrased, reorganised and grammar-corrected into study material; nothing added.

This PDF is shared under the same licence (CC BY-SA 4.0).